Formal Verification of the Snake Game as a State Machine

Clément Yves Pierre Tsedri
clement.tsedri@epfl.ch

Mohamed Taha Guelzim
mohamed.guelzim@epfl.ch

Ozair Faizan
ahmad.faizan@epfl.ch

Abstract

The Snake game, while considered trivial software, presents a complex environment and large state spaces which makes it an excellent candidate for a formal verification case study. In this project, we model the game as a state machine and verify the correctness of its game logic using Stainless, a verification framework for Scala. We define a set of invariants describing a valid playing game state and prove that the game's transition function preserves these invariants along with correct transitions to terminal states (Game Over or Game Won).

Our results demonstrate that modern SMT-based verification tools can effectively guarantee the absence of illegal states in dynamic game environments.

We used the paper [1], which formalizes simple games such as Pac-Man and Chess in ProB, as inspiration for our formalization.

1 Introduction

Formal verification is traditionally associated with safety-critical systems; however, applying these methods to game development offers a unique opportunity to study state-based reasoning and invariant preservation in a system with immediate visual feedback and intuitive rules.

The Snake game involves a player-controlled snake moving on a grid, growing as it consumes food, and dying if it collides with the walls or itself. While traditionally a real-time game, our implementation focuses on a turn-based version, meaning the game state updates only when the player provides input (although the game is playable in real-time too). Despite its apparent simplicity, the state space is vast, making exhaustive testing impossible.

In this project, our aim is to verify that our implementation of Snake never enters an undefined or illegal state during execution. We model the game in Scala using immutable data structures and use the Stainless framework to prove inductive invariants (safety properties).

Our contributions are as follows:

- We formalize Snake as a finite state machine in Scala, separating control flow from the game logic.
- We define inductive invariants, encompassing geometric bounds and topological consistency.
- We provide machine-checked proofs that the game transition function preserves these invariants and that the food generation algorithm correct.

2 Model and Game Logic

The game is implemented as a finite state machine, the implementation is available on Github, the state is captured by

```
case class GameState(
    snake: List[Position],
    food: Option[Position],
    direction: Direction,
    status: GameStatus,
    config: GameConfig,
    stateNumber: BigInt,
    pendingDirection: Option[Direction]
)
```

where the `status` field determines the high-level mode of the FSM described in Figure 1. The game state is represented using immutable data structures, which simplifies formal reasoning about state transitions by making each transition explicit and side-effect free. The core verification focuses on the `Playing` status, where the logic is non-trivial.

The core logic resides in the function `processGameTick` which advances the game state by one tick while in a `Playing` status. It computes the next head position, checks for collisions, moves the snake, handles wins, and may generate new food. The snake movement is handled as follows: if food was eaten, then the new snake is `nextHead :: oldSnake`; otherwise it is `nextHead :: withoutLast(oldSnake)` where `withoutLast` removes the last element of the list.

The function `def generateFood(state: GameState, seed: BigInt): Position` outputs a grid position where the food will be placed such that it is not occupied by the snake. It uses `seed` which is a pseudo-random number.

Verification of the above two functions is the core of our project.

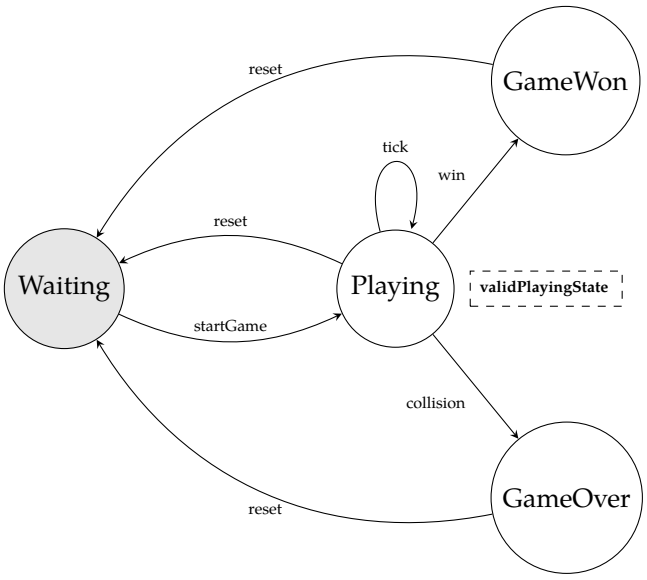


Figure 1: High-level state diagram of the game.

3 Specifications and Invariants

We require that every state in `Playing` status satisfies

```
def validPlayingState(state: GameState): Boolean =
  state.status == GameStatus.Playing &&
    0 < state.snake.length &&
```

```
state.snake.length < state.gridWidth * state.gridHeight &&
validPendingDirection(state) &&
allWithinBounds(state.snake, state.gridWidth, state.gridHeight) &&
noSelfIntersection(state.snake) &&
contiguous(state.snake)
```

which serves as the central invariant, that states:

- The snake length is strictly positive and less than the total grid size $W \times H$. Note: $W, H \geqslant 3$.
- The pending direction is never the opposite of current direction unless the snake length is 1.
- All snake positions are within the grid boundaries.
- The snake has no duplicate positions.
- Each segment of the snake is adjacent to the next segment.

Furthermore, we require `processGameTick` to satisfy `validTransition` which describes valid state transitions. These conditions ensure that the game remains consistent:

- The snake length never decreases and grows by at most one.
- The tail of the new snake must be a subsequence of the previous snake (movement preserves body).
- Reaching `GameOver` is equivalent to `nextHead` colliding with the walls or with the snake.
- Reaching `GameWon` is equivalent to the snake filling the grid.

Finally, for a `Playing` state, the `generateFood` function can always return a valid position to place the food. This relies on the invariant that the length of the snake is strictly less than the size of the grid. Proving that an empty spot exists and can be correctly mapped from the random seed is non-trivial.

4 Verification of Game Logic

4.1 Initial Playing State is Valid

The verification begins with the transition from `Waiting` to `Playing`. The function `initializeGame` places a snake of length 1 in the center of the grid, heading right with no pending direction. The invariant `validPlayingState` is satisfied, since the snake consists of a single position, self-intersection is impossible, and the contiguity property holds trivially for the base case of a single element list. The bounds check is satisfied by the construction of the initial position at $(W/2, H/2)$.

4.2 Preservation of Invariants

In order to prove that `validPlayingState` is an inductive invariant we need to show that `processGameTick` preserves the invariant when the game does not end.

To verify this, we implemented lemmas in `ListUtilsProperties` regarding the `withoutLast` operation used for the snake movement. We proved inductively that if a list `s` is contiguous, then `withoutLast(s)` is also contiguous. The proof relies on list induction: for the recursive step `Cons(h1, t @ Cons(h2, _))`, the adjacency of `h1` and `h2` is preserved regardless of the regardless of the removal of the last element of `t`.

Similarly, we proved that removing the last element preserves the `noSelfIntersection` and `allWithinBounds` properties. Specifically, `withoutLastNoSelfIntersection` leverages the fact that `withoutLast(s)` is a subsequence of `s`. Since the original snake has no duplicates, any subsequence inherently satisfies the non-intersection property.

Stainless successfully verifies that adding a valid `newHead` (checked against collision) to this valid tail results in a new snake that is contiguous, non-intersecting and within boundaries.

Furthermore, we proved that the length of the snake grows by at most 1, therefore, the length bounds are preserved (if we stay in the `Playing` status).

As for `validPendingDirection`, the `queueDirectionChange` function ensures that the predicate is verified.

4.3 Transition Safety

Beyond the structural invariants of the snake, we verify the transition logic itself via `validTransition`. A critical safety property is that the snake body does not teleport or change shape arbitrarily. We formalize this by requiring `ListSpecs.subseq(next.snake.tail, curr.snake)`. This ensures that the body of the snake in the next state is a subsequence of the snake in the previous state. This required 3 simple lemmas: `subseqOfSelf` which proves that `s: List[T]` is a subsequence of itself, used when the snake does not change; `subseqTail` which is provided by `ListSpecs`, used when the snake eats the food; and `withoutLastSubseq`, used when the snake does not eat the food.

Furthermore, Stainless successfully verifies that the snake length is bounded by `curr.length <= next.length <= curr.length + 1`, and reaching `GameOver` and `GameWon` conditions without any further help.

4.4 Food Generation Safety

The verification of the `generateFood` function ensures that for any valid playing state, it is always possible to find a grid position that is not occupied by the snake. The verification relies on proving that the set difference between all grid positions and the snake body is non-empty, and the generated position must be within the grid boundaries and not collide with the snake.

We first established that the grid construction, which maps a range of integers to positions, is injective and produces no duplicates. This is proven in `gridInjection`, showing that the mapping $i \mapsto (i\%w, i/w)$ is an injection from the integer range to grid coordinates. Consequently, the list returned by `grid(width, height)` contains unique elements, since we proved, range of integers have no duplicates in `rangeNoDupliates` and mapping a list with no duplicates with the above injective function gives no duplicates in `gridInjectionNoDuplicate`, in `GridProperties`.

To ensure valid selection, we must prove that `grid -- snake` is non-empty. The crux of this proof lies in the lemma `noDuplicateFilterLength` in `ListProperties`. This lemma proves that if a list ℓ_2 (the grid) has no duplicates, and ℓ_1 (the snake) is a strictly shorter list, then the length of $\ell_2 - \ell_1$ is at least $|\ell_2| - |\ell_1|$. Since the invariant `validPlayingState` guarantees that the snake's length is strictly less than the total grid size ($W \times H$), the precondition holds, and the set of empty positions is guaranteed to be non-empty.

To prove the `isWithinBounds` condition, we rely on the properties of the full grid and subsequence preservation. First, we prove, `rangeMappingWithinBound`, that every element in the full grid list, generated by `grid(width, height)`, satisfies the `isWithinBounds` predicate. Next, we determine the list of potential food positions by computing `grid -- snake`, which we prove is a valid subsequence of `grid` using the `removeSubseq` lemma. Finally, we apply the `subseqPreservesProperty` lemma, which asserts that if a property holds for all elements of a list, it holds for all elements of its subsequence. Consequently, since all positions in `grid` are within bounds, any position selected from `grid -- snake` is guaranteed to be within bounds.

With the boundary condition established, the collision freedom condition `!hasCollision(pos)` follows directly. The `hasCollision` predicate checks if a position is out of bounds or contained in the snake. Since we proved the position is within bounds, and it is selected from `grid -- snake`

(which explicitly excludes snake positions), Stainless automatically proves `!hasCollision(pos)` once we instantiation the for all with `ListSpecs.applyForAll`.

5 Results and Metrics

The formalization was successfully verified by Stainless. On a standard modern CPU, the full verification suite completes in approximately 3 seconds without caching. In the GitHub CI/CD pipeline, the verification takes approximately 10 seconds.

The verified code-base consists of approximately 400 lines of Scala code, split roughly in one third for the game implementation and two thirds for the proof infrastructure. A significant portion of the effort was dedicated to auxiliary lemmas. This demonstrates a typical ratio in formal verification where the time spent on proofs and the proof code size exceeds the implementation's.

6 Conclusion

In this project, we demonstrated that formal verification methods can be effectively applied to state-based games like Snake. By modeling the game as a finite state machine and defining invariants for the playing state, we proved that the game logic is sound, free of runtime errors, and consistent with the rules of the game. We utilized structural induction to prove properties concerning list manipulations such as contiguity, subsequence preservation and set theory logic to guarantee the safety of food generation. This work highlights that tools like Stainless allow for the rigorous verification of non-trivial state spaces, ensuring software correctness beyond what is achievable through traditional testing. This project also highlights the limitations of automated verification tools, as several non-trivial properties, particularly those involving list manipulation and grid reasoning, required carefully designed auxiliary lemmas.

7 Further Work

While this project establishes the correctness of the core Snake game logic, several meaningful extensions could be explored in future work.

Stronger Properties and Liveness Our verification focuses primarily on safety properties, ensuring that the game never reaches an invalid state. A natural extension would be to reason about liveness properties. For example, one could prove that from any valid `Playing` state, the game eventually reaches a terminal state (`GameOver` or `GameWon`) under reasonable assumptions on player input. Another direction would be to formally characterize determinism, i.e., prove that for a given state and input, the transition function always produces a unique next state.

Formally Verified Solver A natural extension is the implementation of a formally verified solver. Specifically, when one of the dimensions is even, there always exists a Hamiltonian cycle which the snake can follow to guarantee a win.

Extended Game Rules The current model assumes a single food item, empty boards, and hard boundaries. The framework could be extended to support richer game mechanics, such as multiple simultaneous food items, static obstacles, or wrap-around borders. Each of these features would require new invariants and additional reasoning about collision detection and food placement, providing a more challenging verification task while preserving the same state-machine structure.

Scalability and Automation Although verification time remains reasonable for this project, the number of auxiliary lemmas required for list manipulation and grid reasoning highlights the limits of automation. Future work could investigate alternative encodings of the snake body or grid, or compare Stainless with other verification tools to evaluate proof effort, expressiveness, and scalability.

Executable Refinement Finally, one could establish a refinement relation between the verified model and a real-time or graphical implementation of the game. Proving that the executable version refines the verified state-machine specification would strengthen the connection between formal guarantees and deployed software.

8 References and AI usage

- Clément
  - Attempts were made to use ChatGPT to generate direct proofs; however, this was not successful. ChatGPT was more useful for interpreting Stainless verification output when proofs were invalid, but it was unable to infer concrete solutions to the reported issues.
- Mohamed Taha
  - LLMs were helpful for frontend development. They were used to implement the dark theme and the debug mode, where snake cells are numbered.
  - The initial WebSocket implementation (both frontend and backend) was buggy, but LLMs helped fix the issues after a few iterations.
  - LLMs were not very useful for Stainless: they frequently hallucinated code or attempted to use Scala instead of Stainless.
  - The presentation was partially created using AI (Copilot), which efficiently extracted relevant code blocks and integrated them into the LaTeX slides. It also gave good suggestions on how our presentation related to the codebase, since it had access to it.
- Ozair
  - When migrating the game logic from Scala standard library to Stainless, we had issues with the JSON serializer/deserializer. The library provided predefined functions for standard Scala List and Option but not for the Stainless's. A series of simple prompts with the code, error, and explanation of why the issue occurred solved the issue.
  - Updating the visuals of the game (which I did not write) when changing the backend to add/remove features was done using simple prompts describing the required additions/removals which worked surprisingly well.
  - Asking for proofs generally did not work, it usually combined already proven lemmas or gave incomplete proofs that caused Stainless to time-out.

[1] Sebastian Krings and Philipp Körner. "Prototyping Games using Formal Methods". In: *Formal Methods – Fun for Everybody*. Accessed from: https://fmfun.github.io/Papers-2019/Krings-Koerner.pdf. 2019. DOI: 10.1007/978-3-030-71374-4.